

Product Requirements Document (PRD)

EnergiCast — Energy Consumption Forecasting Platform

Document Version: 1.0

Status: Product Definition / Implementation Ready

Product Type: Web-based machine-learning forecasting platform

Primary Domain: Energy Analytics / Time-Series Forecasting

Initial Deployment Model: Dataset-based, batch forecasting

Development Methodology: Agile, 4 one-week sprints

1. Executive Summary

EnergiCast is a role-based web platform that enables organizations to prepare historical energy-consumption data, train and evaluate time-series machine-learning models, publish validated forecasting models, and generate future energy-consumption forecasts through an accessible web interface.

The product separates **model and dataset management** from **forecast consumption**:

- **Administrators** manage datasets, preprocessing configurations, model training, evaluation, and model publication.
- **Users** consume published forecasting capabilities by selecting supported energy types, features, and forecast horizons.

The initial release focuses on historical CSV datasets rather than real-time IoT or smart-meter ingestion.

The product is designed as an end-to-end ML system rather than a standalone prediction script:

Data Ingestion → Validation → Cleaning → Feature Engineering → Training → Evaluation → Model Registry → Forecasting → Visualization

2. Product Vision

Vision Statement

Build a practical and extensible forecasting platform that converts historical energy data into reliable, interpretable forecasts through a controlled machine-learning workflow.

Long-Term Vision

EnergiCast can evolve from a dataset-based forecasting application into an energy intelligence platform supporting:

- Real-time meter ingestion
- Automated retraining
- Model monitoring
- Forecast uncertainty
- Anomaly detection
- Energy demand alerts
- Weather-aware forecasting
- Multi-site forecasting
- Advanced deep-learning models

These capabilities are outside the initial release.

3. Problem Statement

Energy consumption is temporal and influenced by historical patterns, seasonality, weather, and calendar effects.

Organizations often face three practical problems:

1. Raw energy data requires significant preprocessing before it can be used for forecasting.
2. Comparing forecasting models manually is time-consuming and difficult to reproduce.
3. Forecasting outputs are often trapped inside notebooks or scripts rather than exposed through a usable application.

EnergiCast addresses these problems by creating a structured pipeline from dataset ingestion to forecast visualization.

4. Product Goals

4.1 Primary Goals

1. Provide a reliable workflow for historical energy dataset ingestion.
2. Automate dataset validation and cleaning.
3. Support time-series-specific feature engineering.
4. Train multiple suitable forecasting models.
5. Evaluate models using standardized metrics.
6. Maintain model and dataset metadata.
7. Allow Admins to publish validated models.
8. Allow Users to generate forecasts without direct ML interaction.
9. Visualize historical and predicted consumption clearly.
10. Maintain strict Admin/User access separation.

4.2 Secondary Goals

- Make the ML workflow reproducible.

- Keep the architecture modular.
 - Support additional forecasting algorithms later.
 - Maintain a clean separation between frontend, API, database, and ML components.
 - Make the system deployable using conventional cloud infrastructure.
-

5. Non-Goals — V1

The following are explicitly excluded from the first release:

- Real-time smart-meter streaming
- Kafka/MQTT ingestion
- Continuous online learning
- Automatic production retraining
- Kubernetes
- Distributed ML training
- Multi-region infrastructure
- Enterprise SSO
- Billing/subscription management
- Native mobile application
- Fully autonomous model selection
- Deep learning as a mandatory forecasting method

This keeps the first release technically achievable within the four-week development plan.

6. Target Users

6.1 Administrator

The Administrator is responsible for maintaining the forecasting system.

Responsibilities

- Upload datasets
- Validate datasets
- Configure target and timestamp columns
- Configure available features
- Initiate training
- Review model metrics
- Publish models
- Manage supported forecasting configurations

Technical Skill

Moderate technical knowledge is expected.

6.2 End User

The End User consumes published forecasting functionality.

Responsibilities

- Select an energy type
- Select available forecasting features
- Select forecast horizon
- Generate forecasts
- View forecast charts
- Review previous forecasts

The User does not manage datasets or train models.

7. User Personas

Persona A — Data/ML Administrator

Goal: Maintain a high-quality forecasting pipeline.

Needs:

- Dataset validation
- Training controls
- Model comparison
- Metrics
- Version tracking
- Model publication

Pain Point:

Manual notebook-based workflows are difficult to reproduce and expose to other users.

Persona B — Energy Analyst / Consumer

Goal: Obtain future energy-consumption estimates.

Needs:

- Simple configuration
- Forecast visualization
- Historical comparison
- Forecast history

Pain Point:

Using ML code directly requires unnecessary technical knowledge.

8. Product Scope

8.1 In Scope

Data

- CSV upload
- Dataset validation
- Data cleaning
- Timestamp processing
- Dataset metadata
- Dataset version information

ML

- Time-series EDA
- Feature engineering
- Chronological splitting
- Baseline models
- Classical forecasting models
- Machine-learning forecasting models
- Hyperparameter tuning
- Evaluation
- Model persistence

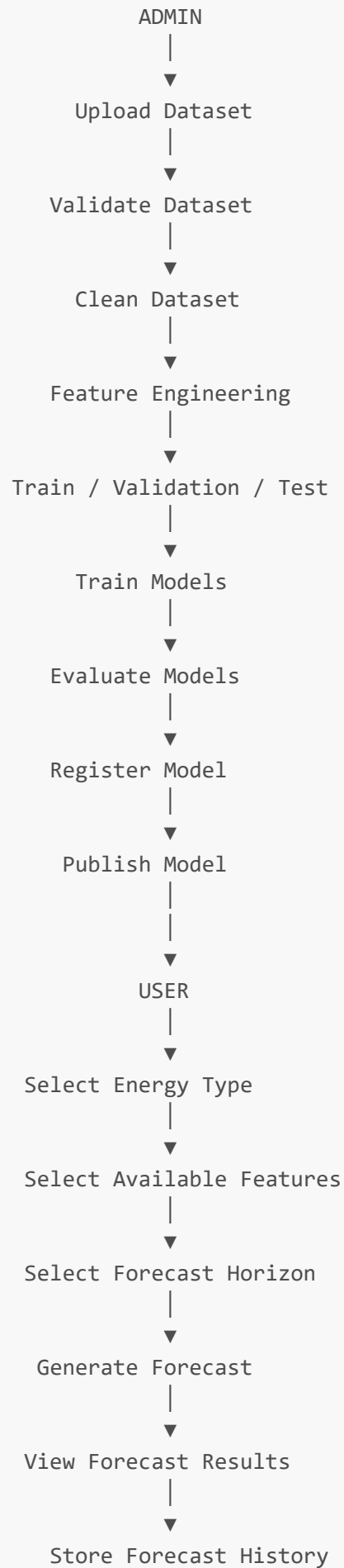
Application

- Authentication
- Role-based access control
- Admin dashboard
- User dashboard
- Forecast configuration
- Forecast visualization
- Forecast history

Infrastructure

- PostgreSQL
- REST API
- Frontend/backend separation
- Environment-based configuration
- Cloud deployment

9. Core Product Workflow



10. Functional Requirements

FR-01 — Authentication

The system shall provide secure authentication for Admin and User accounts.

Acceptance Criteria

- User can register.
 - User can log in.
 - Invalid credentials are rejected.
 - Passwords are not stored in plaintext.
 - Authenticated sessions/tokens are validated by the backend.
-

FR-02 — Role-Based Access Control

The system shall distinguish between Admin and User roles.

Acceptance Criteria

- Admin can access Admin functionality.
 - User cannot access Admin endpoints.
 - Backend authorization is enforced independently of frontend route protection.
 - Unauthorized requests return an appropriate HTTP error.
-

FR-03 — Dataset Upload

The Admin shall be able to upload a supported CSV dataset.

Acceptance Criteria

- Only permitted file types are accepted.
 - File-size limits are enforced.
 - Uploaded files receive server-controlled storage paths.
 - Dataset metadata is recorded.
 - Invalid files generate understandable error messages.
-

FR-04 — Dataset Validation

The system shall validate uploaded datasets before training.

Validation Areas

- Required columns
- Timestamp validity
- Target-column existence
- Numeric target values

- Missing values
- Duplicate timestamps
- Data ordering
- Date range
- Frequency
- Record count
- Data sufficiency

Acceptance Criteria

A dataset must not proceed to training when critical validation requirements fail.

FR-05 — Dataset Cleaning

The system shall apply configured cleaning operations.

Possible Operations

- Timestamp parsing
- Sorting
- Duplicate handling
- Numeric conversion
- Missing-value handling
- Invalid-row handling
- Frequency alignment

All transformations should be reproducible.

FR-06 — Feature Engineering

The system shall generate time-series features appropriate for the dataset frequency.

Potential Features

Calendar:

- Hour
- Day
- Day of week
- Week
- Month
- Quarter
- Weekend indicator

Lag:

- Lag 1
- Lag 2
- Lag 24

- Lag 48
- Lag 168

Rolling:

- Rolling mean
- Rolling standard deviation
- Other appropriate rolling statistics

The exact windows must be determined from the dataset's temporal frequency.

Acceptance Criteria

- Features do not use future information.
- Feature generation is deterministic for the same input.
- Training and inference use compatible feature definitions.

FR-07 — Dataset Splitting

The system shall split time-series data chronologically.

Example:



Random shuffling shall not be used for the primary forecasting evaluation.

FR-08 — Baseline Models

The system shall support simple baselines.

Required

- Naive forecast
- Seasonal Naive where seasonality is applicable

Baselines establish a minimum reference level for model performance.

FR-09 — Forecasting Models

The initial model catalogue should include:

Model	Purpose
ARIMA	Univariate time-series forecasting
SARIMA	Seasonal univariate forecasting
Random Forest	Nonlinear feature-based forecasting
XGBoost	Gradient-boosted feature-based forecasting

Models should be applied according to dataset characteristics rather than forcing every model onto every dataset.

FR-10 — Model Training

The Admin shall be able to initiate model training.

The training pipeline shall record:

- Dataset version
 - Model type
 - Hyperparameters
 - Training timestamp
 - Feature configuration
 - Metrics
 - Artifact location
 - Training status
-

FR-11 — Hyperparameter Tuning

The system should support controlled hyperparameter tuning for applicable ML models.

Requirements

- Tuning must use validation data.
 - The test set must remain untouched until final evaluation.
 - Search configuration should be reproducible.
 - Selected parameters must be stored.
-

FR-12 — Model Evaluation

The system shall calculate forecasting metrics.

Required Metrics

- MAE
- RMSE

Optional Metric

- MAPE, only when mathematically appropriate.

Metric Definitions

```
MAE = mean(|actual - predicted|)
```

```
RMSE = sqrt(mean((actual - predicted)^2))
```

The system should display metrics with their corresponding evaluation dataset.

FR-13 — Model Registry

The system shall maintain model metadata and lifecycle state.

Example Lifecycle

```
TRAINING
  ↓
EVALUATED
  ↓
APPROVED
  ↓
PUBLISHED
```

A failed training run must not replace an existing published model.

FR-14 — Model Publication

Only an Admin shall be able to publish a model.

A published model must have:

- Valid artifact
 - Associated dataset version
 - Feature configuration
 - Evaluation metrics
 - Model version
 - Publication status
-

FR-15 — Forecast Generation

The User shall be able to request a forecast using a published model.

Inputs

- Energy type
- Supported feature configuration
- Forecast horizon

Output

- Future timestamps
- Predicted consumption
- Model metadata
- Forecast request metadata

FR-16 — Forecast Visualization

The system shall visualize:

- Historical consumption
- Forecast values
- Forecast horizon
- Time axis
- Consumption axis

Optional:

- Confidence/prediction intervals when supported
- Actual-vs-predicted comparison
- Key forecast statistics

FR-17 — Forecast History

The system shall store forecast requests and results.

Users should be able to review previous forecast operations.

11. Non-Functional Requirements

NFR-01 — Performance

For the initial target of a small number of concurrent users:

- Standard dashboard API calls should normally respond within a few seconds.
- Forecast inference should be responsive after the model is loaded.
- Model training may be asynchronous if execution time becomes significant.

Training latency is not treated as normal interactive API latency.

NFR-02 — Reliability

- Failed training jobs must not corrupt published models.
 - Invalid datasets must be rejected before model training.
 - Forecast requests must fail gracefully when no valid published model exists.
-

NFR-03 — Security

- Password hashing required.
 - Secrets stored outside source code.
 - Backend authorization required.
 - Uploaded file paths must be controlled by the server.
 - File-size and file-type restrictions required.
 - CORS must be restricted for production.
-

NFR-04 — Maintainability

The system shall separate:

```
Frontend
Backend API
Data Processing
ML Training
Forecast Inference
Database
Storage
```

ML logic should not be embedded directly inside API route handlers.

NFR-05 — Reproducibility

A model should be reproducible from:

```
Dataset Version
+
Feature Configuration
+
Preprocessing Configuration
+
Model Type
+
Hyperparameters
```

NFR-06 — Scalability

The architecture should allow future replacement/addition of:

- Background workers
- Redis
- Object storage
- Distributed training
- Streaming ingestion

These are not required for V1.

12. Data Requirements

12.1 Initial Dataset Sources

The current project uses dataset files obtained from a time-series dataset source.

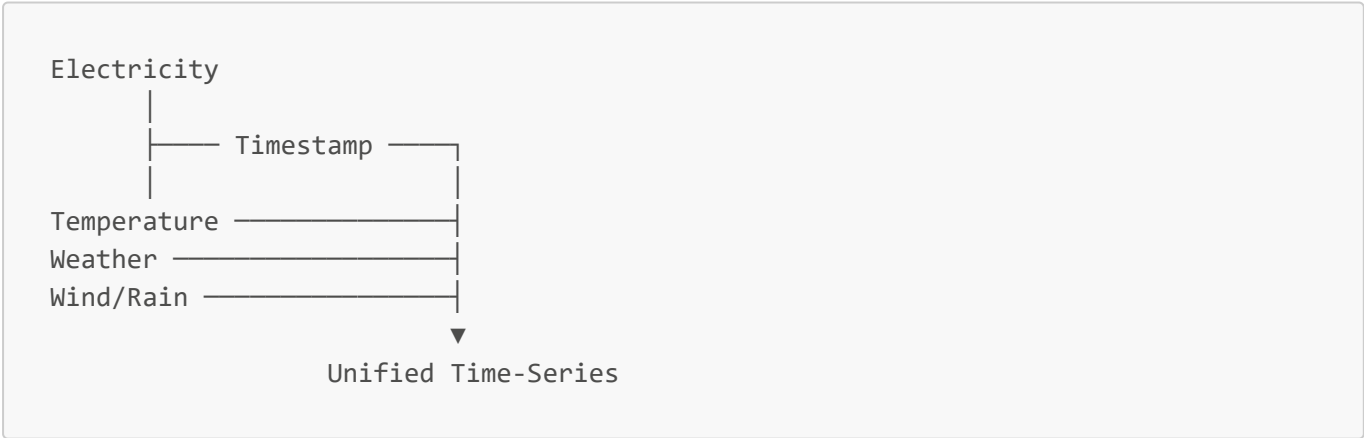
Current files:

```
electricity.csv
weather.csv
temperature.csv
wind_rain.csv
```

The actual schema must be inspected before implementation assumptions are finalized.

12.2 Dataset Relationship

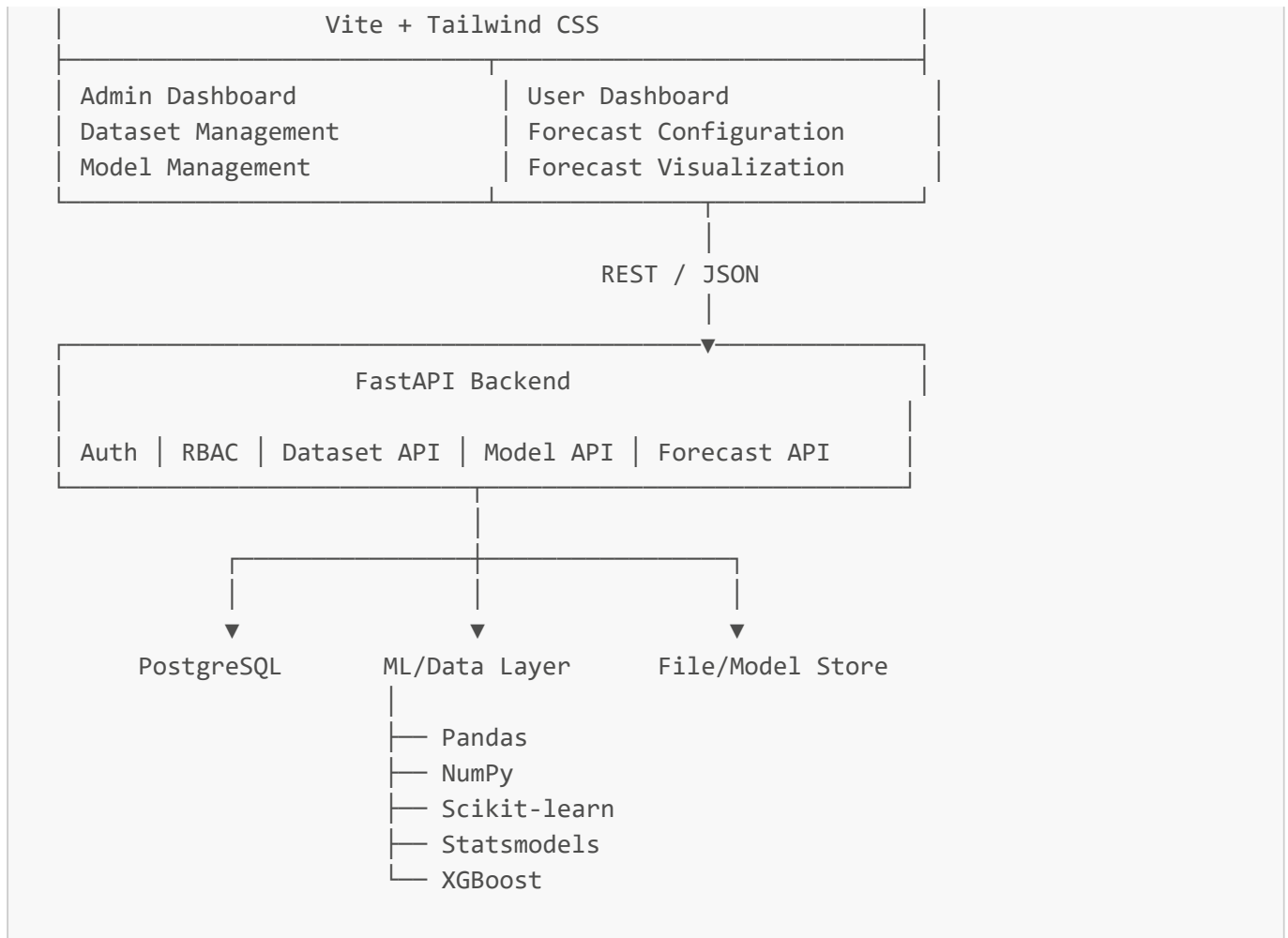
Where timestamps are compatible:



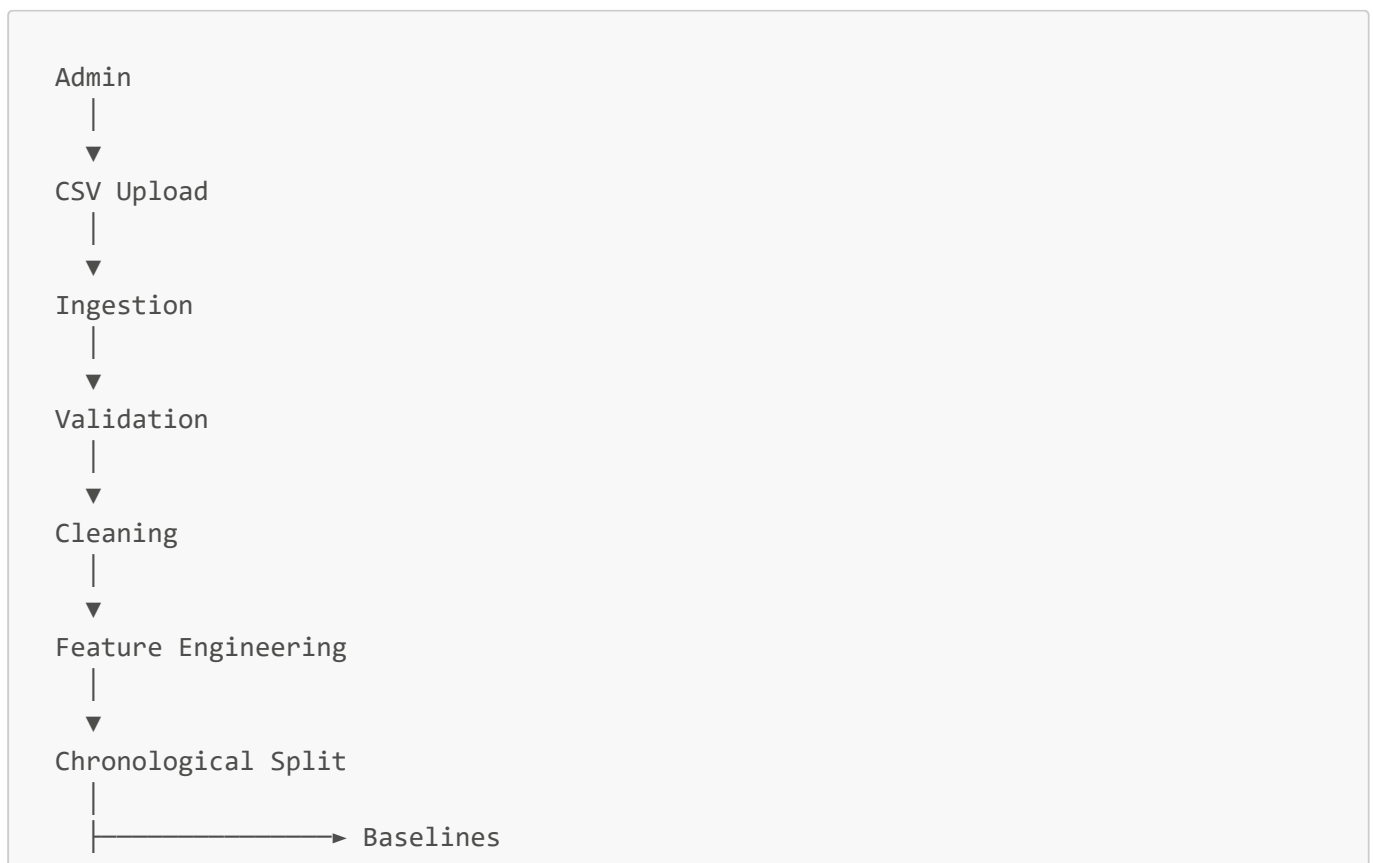
If the files have different frequencies or timestamps, the system shall explicitly perform resampling/alignment.

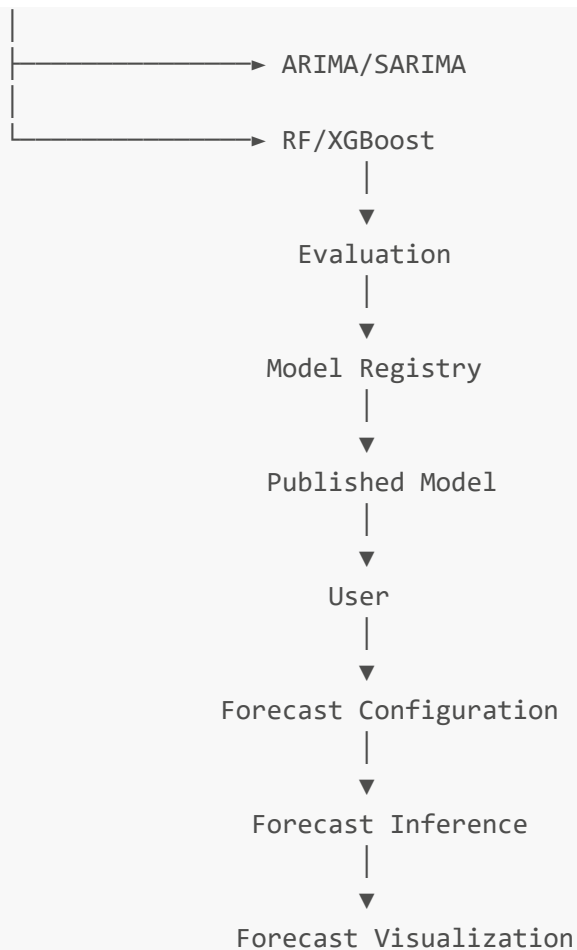
13. Product Architecture





14. Data Flow





15. Database Requirements

Users

```
users
-----
id
name
email
password_hash
role
created_at
updated_at
```

Datasets

```
datasets
-----
id
name
```



```
energy_type
version
file_path
timestamp_column
target_column
frequency
status
uploaded_by
created_at
updated_at
```

Features

```
features
-----
id
dataset_id
name
display_name
feature_type
enabled
```

Models

```
models
-----
id
dataset_id
model_name
version
hyperparameters
metrics
artifact_path
status
created_at
```

Forecasts

```
forecasts
-----
id
user_id
dataset_id
model_id
```

```
horizon
created_at
```

Forecast Values

```
forecast_values
-----
id
forecast_id
timestamp
predicted_value
actual_value
```

16. API Requirements

Authentication

```
POST /api/auth/register
POST /api/auth/login
```

Dataset

```
GET /api/datasets
POST /api/admin/datasets
GET /api/admin/datasets/{id}
```

Model

```
POST /api/admin/models/train
GET /api/admin/models/{dataset_id}
POST /api/admin/models/{id}/publish
```

Forecast

```
POST /api/forecast
GET /api/forecasts
GET /api/forecasts/{id}
```

All `/api/admin/*` routes must require Admin authorization.

17. UX Requirements

17.1 Design Principles

The interface should be:

- Clear
- Data-oriented
- Responsive
- Accessible
- Consistent
- Minimal in unnecessary interaction complexity

17.2 Admin Dashboard

Primary sections:

```
Dashboard
├─ Dataset Status
├─ Recent Uploads
├─ Training Jobs
├─ Model Metrics
└─ Published Models
```

17.3 User Dashboard

Primary sections:

```
Dashboard
├─ Energy Type
├─ Available Features
├─ Forecast Horizon
├─ Generate Forecast
└─ Recent Forecasts
```

17.4 Forecast Screen

Should show:

1. Forecast configuration
2. Historical consumption
3. Forecasted consumption
4. Time range

5. Model used
6. Evaluation information where relevant
7. Forecast table
8. Export option if implemented

18. Error Handling

The system should provide actionable errors.

Dataset Error

```
Dataset rejected:  
Timestamp column contains invalid values.  
Please correct the dataset and upload it again.
```

Model Error

```
Training failed:  
The selected dataset does not contain sufficient observations  
for the configured seasonal period.
```

Forecast Error

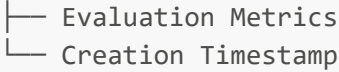
```
Forecast unavailable:  
No published model exists for this energy configuration.
```

Avoid exposing stack traces or internal infrastructure details to users.

19. Model Governance

Every published model should be traceable to:

```
Model Version  
|  
├─ Dataset Version  
├─ Feature Configuration  
├─ Preprocessing Configuration  
└─ Hyperparameters
```



```
graph LR; A[Evaluation Metrics] --> B[ ]; C[Creation Timestamp] --> B;
```

This allows the team to understand exactly what produced a forecast.

20. Testing Strategy

Unit Tests

- Cleaning functions
- Validation functions
- Feature generation
- Metrics
- Model utilities

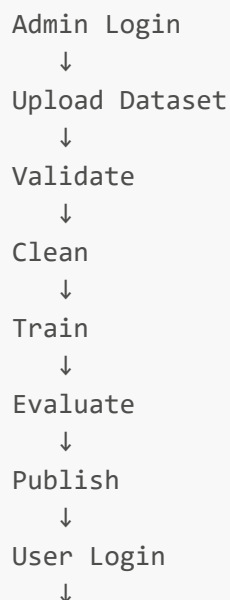
Integration Tests

- Dataset upload → processing
- Training → registry
- Published model → forecast

API Tests

- Authentication
- RBAC
- Dataset APIs
- Training APIs
- Forecast APIs

End-to-End Test



```
graph TD; A[Admin Login] --> B[Upload Dataset]; B --> C[Validate]; C --> D[Clean]; D --> E[Train]; E --> F[Evaluate]; F --> G[Publish]; G --> H[User Login]; H --> I[ ];
```

```
Configure Forecast
  ↓
Generate
  ↓
Visualize
```

21. Four-Sprint Delivery Roadmap

Sprint 1 — Foundation and Data Pipeline

Deliverables

- Repository
- React frontend
- FastAPI backend
- PostgreSQL
- Environment configuration
- Authentication foundation
- CSV ingestion
- Validation
- Cleaning

Definition of Done

A valid CSV can be uploaded and transformed into a clean, validated dataset.

Sprint 2 — Forecasting Engine

Deliverables

- Time-series EDA
- Feature engineering
- Chronological splitting
- Baselines
- ARIMA/SARIMA
- Random Forest
- XGBoost
- Metrics
- Model persistence

Definition of Done

At least one valid dataset can be used to train and evaluate multiple forecasting approaches.

Sprint 3 — Product Integration

Deliverables

- Admin dashboard
- User dashboard
- RBAC
- Dataset management
- Model registry
- Model publication
- Forecast APIs

Definition of Done

An Admin can publish a model and a User can request a forecast from it.

Sprint 4 — Production Readiness

Deliverables

- Forecast visualization
- Forecast history
- Testing
- Security hardening
- Deployment
- Documentation
- End-to-end validation

Definition of Done

The complete Admin → Model → User → Forecast workflow operates in the deployed application.

22. Success Metrics

For V1, product success should be measured primarily by system functionality and ML validity rather than claiming a universal accuracy target.

Product Metrics

- Dataset ingestion success rate
- Dataset validation success/failure reporting
- Forecast request success rate
- API error rate
- Forecast generation latency
- User workflow completion

ML Metrics

- MAE

- RMSE
- MAPE where appropriate
- Baseline comparison
- Validation vs test performance
- Forecast horizon performance

Quality Criteria

A model should not be considered useful solely because it produces a low numerical error. Its performance should also be compared against appropriate baselines and evaluated on unseen chronological data.

23. Risks and Mitigations

Risk	Impact	Mitigation
Poor dataset quality	High	Validation and cleaning pipeline
Missing values	Medium/High	Explicit missing-value strategy
Data leakage	Critical	Strict chronological feature generation
Insufficient data	High	Dataset sufficiency checks
Overfitting	High	Validation set + baseline comparison
Model mismatch	Medium	Model selection based on data characteristics
Training failure	Medium	Job status + error handling
File storage loss	High	Persistent/object storage in production
Unauthorized Admin access	Critical	Backend RBAC
Feature mismatch at inference	Critical	Persist preprocessing/feature metadata
Model artifact corruption	High	Versioned artifacts and publication states
API failure	Medium	Error handling and logging

24. Observability and Logging

The backend should record structured logs for important operations.

Examples:

```
DATASET_UPLOADED
DATASET_VALIDATED
DATASET_CLEANED
TRAINING_STARTED
TRAINING_COMPLETED
```



```
TRAINING_FAILED  
MODEL_PUBLISHED  
FORECAST_REQUESTED  
FORECAST_COMPLETED  
FORECAST_FAILED
```

Logs should contain useful identifiers without exposing secrets or sensitive data.

25. Deployment Requirements

Environment Variables

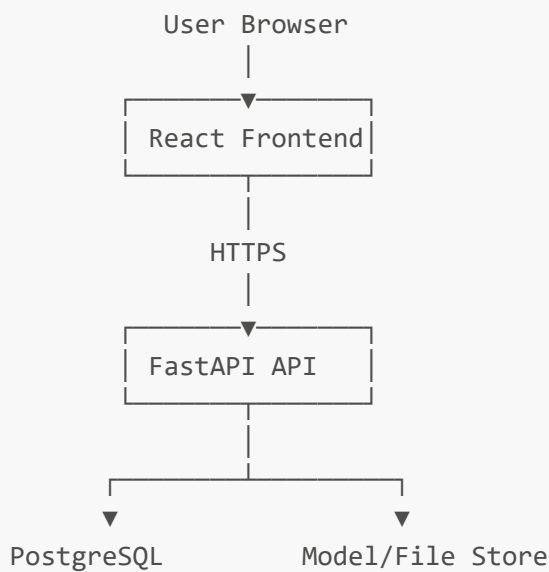
Example:

```
DATABASE_URL=  
SECRET_KEY=  
CORS_ORIGINS=  
MODEL_STORAGE_PATH=
```

Use `.env.example` in source control.

Never commit actual credentials.

Deployment Architecture



26. Future Product Roadmap

Phase 2 — Operational Forecasting

- Scheduled retraining
- Automated model evaluation
- Model drift detection
- Prediction intervals
- Better model monitoring

Phase 3 — Real-Time Energy Intelligence

```
graph TD; A[Smart Meter] --> B[MQTT/Kafka]; B --> C[Stream Processing]; C --> D[Feature Store]; D --> E[Forecast Engine]; E --> F[Dashboard];
```

Phase 4 — Advanced Analytics

- Anomaly detection
- Consumption alerts
- Peak demand detection
- Energy optimization recommendations
- Multi-site forecasting
- Explainable ML

Phase 5 — Advanced Forecasting

- LSTM
- GRU
- Transformer-based models
- Probabilistic forecasting
- Ensemble models

27. Definition of Done — Product Level

EnergiCast V1 is complete when:

- ☐ Admin authentication works.
- ☐ User authentication works.
- ☐ RBAC is enforced on the backend.

- ☐ Admin can upload CSV datasets.
- ☐ Dataset validation works.
- ☐ Dataset cleaning works.
- ☐ Time-series feature engineering works.
- ☐ Chronological train/validation/test splitting works.
- ☐ Baseline models are implemented.
- ☐ Multiple forecasting models are implemented.
- ☐ Model evaluation works.
- ☐ Hyperparameters and model metadata are persisted.
- ☐ Models can be registered and published.
- ☐ User can select supported forecasting options.
- ☐ User can generate forecasts.
- ☐ Forecasts are visualized.
- ☐ Forecast history is stored.
- ☐ Critical workflows have automated tests.
- ☐ Production configuration is secure.
- ☐ Deployment is functional.
- ☐ End-to-end workflow has been verified.

28. Final Product Experience

Admin

```
Login
↓
Admin Dashboard
↓
Upload Dataset
↓
Validate
↓
Clean
↓
Configure
↓
Train Models
↓
Compare Metrics
↓
Publish Model
```

User

```
Login
↓
```

```
User Dashboard
↓
Select Energy Type
↓
Select Available Features
↓
Select Forecast Horizon
↓
Generate Forecast
↓
View Historical + Predicted Consumption
↓
Save/View Forecast History
```

29. Product Principle

The core principle of EnergiCast is:

Do not treat forecasting as a single model-training task. Treat it as a controlled data-to-decision pipeline.

The product therefore prioritizes:

1. Data quality
2. Temporal correctness
3. Reproducibility
4. Model evaluation
5. Controlled publication
6. Clear visualization
7. Role-based access
8. Extensibility

This structure gives EnergiCast a foundation that can move from an academic forecasting project toward a production-oriented energy analytics system without unnecessarily introducing infrastructure complexity into V1.